

The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch05_regression

AYuSh

Contents

Chapter 5: Regression..... 3

Chapter 5: Regression

Regression is where interviewers go to find out whether your ML knowledge has foundations or just a surface. Linear regression looks like the easiest model in the book — one line of sklearn — which is exactly why the questions about it are hard: "derive the normal equation", "what assumptions are you making, and which ones actually matter for prediction?", "why does Lasso produce zeros and Ridge doesn't?", "why is logistic regression called regression when it classifies?". Each of these has a crisp, complete answer, and interviewers use them to separate candidates who have derived things once from candidates who have only imported them. Regression also carries an outsized share of the field's conceptual weight: gradient descent, regularization, maximum likelihood, and the link between loss functions and probabilistic assumptions all appear here first and reappear everywhere — the last layer of nearly every neural network in this book is a linear or logistic regression wearing learned features.

This chapter builds the full ladder: the linear model and its assumptions (separated, carefully, into what prediction needs versus what inference needs), the two ways to fit it — the closed-form normal equation and gradient descent — and exactly when each wins, polynomial regression as the gateway to capacity control, the regularized family (Ridge, Lasso, Elastic Net) with the geometry that explains their different behavior, logistic regression from log-odds to decision boundary to cross-entropy gradient, and finally Generalized Linear Models, the framework that reveals linear, logistic, and Poisson regression as one idea with three link functions.

Linear regression: the model and its assumptions

Linear regression predicts a continuous target as a weighted sum of features:

$$y = Xw + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Here X is the $n \times d$ design matrix (one row per example, one column per feature, plus a column of ones for the intercept), w the weight vector, and ε the noise — everything about y that the features do not determine. The model's claim is modest and precise: the *conditional mean* of the target is a linear function of the features, $\mathbb{E}[y \mid x] = w^\top x$. Each weight w_j reads as "the expected change in y per unit change in x_j , holding the other features fixed" — a sentence worth rehearsing, because the *holding others fixed* clause is where interview traps live (it is exactly the clause that multicollinearity destroys).

"Linear" means linear *in the parameters*, not in the raw inputs. $y = w_1x + w_2x^2 + w_3\log x$ is still linear regression — the features are nonlinear transforms, but the model is a linear combination of them, and everything in this chapter applies unchanged. This one observation is the entire mechanism of polynomial regression, splines, and interaction

features, and stating it early in an interview signals you know where the model's actual boundary is: it cannot learn which *transforms* to apply; it can only weight the ones you give it. (Neural networks, Chapter 12, are the machine that learns the transforms too.)

The classical assumptions. Textbooks list them as a block; strong candidates split them by what they are *for*. The four:

1. **Linearity:** $\mathbb{E}[y | x]$ really is linear in the supplied features. If it isn't, the model is biased no matter how much data arrives (Chapter 4's high-bias regime).
2. **Independence of errors:** the ε_i are independent across observations. Violated by time series (autocorrelation), grouped data (multiple rows per user), and spatial data.
3. **Homoscedasticity:** constant error variance σ^2 at every x . Violated when spread grows with the prediction — incomes, prices, counts.
4. **Normality of errors:** $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

The split that earns interview points: **for pure prediction, only linearity truly matters** (plus enough independence that your train/test protocol is honest — Chapter 4's leakage discussion). OLS still produces the best linear approximation to the conditional mean under heteroscedastic or non-normal errors; the *point predictions* are fine. What breaks under violations 2–4 is **inference**: standard errors, confidence intervals, p-values, and any "is this coefficient significant?" claim. Under heteroscedasticity the textbook standard-error formula is simply wrong — Listing 8 measures a nominal 95% confidence interval covering the truth only 83% of the time — and under correlated errors it is usually far too optimistic, because n correlated observations carry the information of far fewer independent ones. Normality is the least important in practice: for large n the CLT (Chapter 1) rescues coefficient inference anyway; it matters mainly for prediction *intervals* and small samples.

There is a fifth, often-unstated assumption: **no perfect multicollinearity** — no feature is an exact linear combination of others, which $X^\top X$ needs to be invertible. Near-collinearity is legal but toxic to interpretation, and gets its own treatment below and in Listing 7.

Diagnostics are how the assumptions are checked in practice, and naming them makes answers concrete: plot residuals against fitted values (a curve means nonlinearity; a funnel means heteroscedasticity); plot residuals in time order for autocorrelation (or Durbin-Watson); a Q-Q plot of residuals for normality. The universal message of residual plots: a well-specified model's residuals should look like structureless noise — any pattern left in them is signal the model missed.

Fitting the model: OLS, the normal equation, and gradient descent

Ordinary least squares chooses the weights minimizing the mean of squared residuals:

$$\mathcal{L}(w) = \frac{1}{n} \|y - Xw\|_2^2$$

Why squared error, and not absolute error or something else? Three converging justifications, any of which an interviewer may probe. *Statistical*: under the Gaussian-noise model, maximizing the likelihood of the data is exactly minimizing squared error — write the Gaussian density for each y_i , take logs, and the exponent delivers the sum of squares (MLE, Chapter 1). Squared loss is not an arbitrary choice; it is the Gaussian assumption in disguise, which is also why it is outlier-sensitive: a Gaussian says huge errors are astronomically rare, so the fit contorts itself to shrink them. *Analytical*: the loss is smooth and convex with a closed-form minimizer. *Decision-theoretic*: the function minimizing expected squared error is the conditional mean $\mathbb{E}[y | x]$ — squared loss is what you use when you want the mean. (Absolute loss targets the conditional *median* — more robust, no closed form; that trade is a classic question.)

The normal equation. Set the gradient to zero. Expanding $\mathcal{L}(w) = \frac{1}{n} (y - Xw)^\top (y - Xw)$ and differentiating with respect to w (matrix calculus from Chapter 1):

$$\begin{aligned}\nabla_w \mathcal{L} &= -\frac{2}{n} X^\top (y - Xw) = 0 \Rightarrow X^\top Xw = X^\top y \\ \hat{w} &= (X^\top X)^{-1} X^\top y\end{aligned}$$

Convexity guarantees this stationary point is the global minimum. The geometry is worth one interview sentence: $X\hat{w}$ is the orthogonal projection of y onto the column space of X — the fitted values are the closest point to y reachable by any linear combination of the features, and the residual vector is perpendicular to every feature column ($X^\top (y - X\hat{w}) = 0$ is literally the orthogonality statement).

A tiny worked example makes the machinery concrete. Points $(0, 1), (1, 3), (2, 5)$; model $y = b + wx$. With the intercept column included, $X^\top X$ is the 2×2 matrix with rows $(3, 3)$ and $(3, 5)$, and $X^\top y = (9, 13)$. The normal equation is then the pair $3b + 3w = 9$ and $3b + 5w = 13$, giving $w = 2, b = 1$ — the exact line through the points, residuals zero, as expected for collinear data.

Normal equation vs gradient descent is a standard compare-and-contrast, and the answer has real numbers in it. Solving the normal equation costs $O(nd^2 + d^3)$ — forming $X^\top X$ dominates at $O(nd^2)$, solving the $d \times d$ system adds $O(d^3)$. Gradient descent costs $O(nd)$ per step:

$$w \leftarrow w - \eta \nabla_w \mathcal{L}$$

So: for small-to-moderate d (thousands), the closed form is exact, deterministic, and effectively instant — use it (sklearn's LinearRegression does, via SVD-based least squares, which is the numerically stable route: it never forms $X^\top X$, whose condition number is the *square* of X 's). For very large d , or data too big for memory, or streaming settings, or the moment the loss stops being squared error (logistic regression has no closed form at all), gradient descent and its stochastic variants are the only game — which is why the entire deep learning stack (Chapter 13) runs on them. The honest

summary: the normal equation is a special-case luxury of squared loss; gradient descent is the general-purpose engine.

Gradient descent brings its own obligations. The learning rate η must respect the curvature: convergence requires $\eta < 2/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue of the Hessian $\frac{2}{\eta}X^\top X$. And the *ratio* $\lambda_{\max}/\lambda_{\min}$ — the condition number — controls the speed: gradient descent zigzags down ill-conditioned (elongated) loss bowls, making progress at the rate of the *worst* direction. Features on wildly different scales produce exactly this elongation, which is the real reason for standardization before gradient-based fitting. Listing 2 measures it: raw features with a $100\times$ scale gap give a condition number of 8,605 and either divergence or a crawl; standardized, the condition number is 1.0 and the same problem converges $10\times$ faster with a $1,000\times$ larger learning rate. Note what scaling does *not* affect: the closed-form solution — OLS predictions are identical under any affine rescaling of features; only the *optimizer* cares.

Polynomial regression

Polynomial regression is linear regression on manufactured features: replace x with $(x, x^2, \dots, x^k)$ and fit exactly as before. Nothing about the fitting changes — same normal equation, same gradients — because the model is still linear in w . What changes is *capacity*, and polynomial degree is the cleanest capacity dial in ML, which is why it is the canonical demonstration of Chapter 4's bias-variance tradeoff and a favorite interview sketch: underfit line, good cubic, wild degree-15 wiggle through every noise point.

Listing 3 runs the sweep on a known cubic truth with noise of variance 9: degree 1 underfits (validation MSE 23.4), degree 3 hits the truth (validation MSE 9.1 — almost exactly the irreducible noise floor, the best any model could do), and degree 15 overfits (training MSE keeps falling to 3.1, validation rises to 12.5). Two readings worth saying out loud in an interview: the validation minimum sitting *at the noise floor* is what "the model has learned everything learnable" looks like; and training error below the noise floor ($3.1 < 9$) is the smoking gun of memorized noise.

Practical mechanics that distinguish candidates who have actually fit one:

- **Scale explosion:** with $x \in [-3, 3]$, the degree-15 feature spans $\pm 14,000,000$; raw polynomial features produce catastrophically ill-conditioned $X^\top X$. Standardize the expanded features (as Listing 3's pipeline does) or use an orthogonal polynomial basis.
- **Multivariate blowup:** the number of polynomial terms in d features of degree k grows combinatorially — degree-3 in 100 features is already 176,851 terms. Polynomial expansion is a univariate/low- d tool; in high dimensions you want kernels (Chapter 6) or learned features (Chapter 12).
- **Extrapolation:** polynomials shoot to $\pm\infty$ outside the training range, and high-degree fits do so violently. Never trust a polynomial model outside the data's support.

- **Degree selection** is hyperparameter tuning: choose by cross-validation (Chapter 4), never by training error, which is monotonically decreasing in degree.

Ridge, Lasso, and Elastic Net

Chapter 4 introduced penalized loss $\mathcal{L}_{reg}(w) = \mathcal{L}(w) + \lambda\Omega(w)$ as a variance-reduction lever. Applied to linear regression, the three canonical penalties define three named models, and the interview questions concentrate on *why their behaviors differ*.

Ridge regression ($\Omega = \|w\|_2^2$) has a closed form — add λ to the diagonal and the normal equation becomes

$$\hat{w}_{ridge} = (X^\top X + \lambda I)^{-1} X^\top y$$

That $+\lambda I$ is doing three jobs at once, and naming all three is a strong answer.

Numerically, it makes the matrix invertible even when features are collinear or $d > n$ — Ridge always has a unique solution where OLS may have none. *Statistically*, it shrinks coefficients toward zero, trading a little bias for a large variance cut, with the biggest shrinkage applied along the directions of least data variance (in the SVD view, each singular direction is damped by $\sigma_i^2/(\sigma_i^2 + \lambda)$ — small singular values, i.e. ill-determined directions, get crushed; well-determined ones pass nearly untouched). *Bayesianly*, it is MAP estimation under a Gaussian prior $w \sim \mathcal{N}(0, \tau^2 I)$, with $\lambda = \sigma^2/\tau^2$ (Chapter 1's MLE-vs-MAP distinction, made concrete).

Lasso ($\Omega = \|w\|_1$) produces exact zeros — it is simultaneously a fitting procedure and a feature selector. The standard geometric explanation, which interviewers explicitly ask for: picture the constraint view, minimizing squared loss subject to $\Omega(w) \leq t$. The L2 ball is a sphere; the elliptical loss contours expand until they *touch* it, and a sphere's surface has no special points — tangency lands at generic (nonzero) coordinates. The L1 ball is a diamond with corners *on the axes*, where coordinates are exactly zero, and expanding ellipses hit corners with high probability. Same story in gradient language: the L2 penalty's pull toward zero is proportional to w_j — it fades as the weight shrinks, so weights approach but never reach zero. The L1 penalty's pull is a constant λ regardless of size; any weight whose data-gradient can't sustain λ of force gets pinned to exactly zero. (Formally, the soft-thresholding update: for orthonormal features,

$\hat{w}_j = \text{sign}(w_j^{OLS}) \cdot \max(|w_j^{OLS}| - \lambda, 0)$ — small OLS coefficients are zeroed outright.)

Lasso's Bayesian reading is a Laplace prior, sharply peaked at zero. Its costs: no closed form (the kink at zero defeats plain gradient descent; solvers use coordinate descent or proximal methods), at most n nonzero coefficients when $d > n$, and unstable selection among correlated features — from a group of near-duplicates it picks one nearly arbitrarily and zeros the rest, and the pick can flip with a re-sampled dataset.

Elastic Net mixes the two, with sklearn's parameterization

$$\mathcal{L}_{enet}(w) = \frac{1}{2n} \|y - Xw\|_2^2 + \lambda \left(\alpha \|w\|_1 + \frac{1-\alpha}{2} \|w\|_2^2 \right)$$

The L1 part gives sparsity; the L2 part stabilizes it, and in particular fixes the correlated-features pathology: near-duplicate features get *shared* weight (the "grouping effect") instead of an arbitrary winner-take-all. Elastic Net is the default recommendation when you want sparsity and features are correlated — which in real data is almost always.

Listing 4 stages all of this on a 20-feature problem with 5 true nonzeros and one nearly-duplicated feature pair: OLS splits the duplicate pair's weight absurdly (0.05 and 6.07 for a true 3-and-3), Ridge splits it evenly (2.86, 2.89) but zeros nothing, Lasso finds 11 of the 15 true zeros but splits the pair unevenly, and Elastic Net gets both properties at once.

Operational rules that interviews reward: **standardize features first** — the penalty compares coefficient magnitudes across features, which is meaningless if features have different units (a coefficient on "meters" is 1000× the same effect on "kilometers", and the penalty would punish the wrong one); **never penalize the intercept** (it encodes the target's baseline level, not model complexity — penalizing it just biases predictions toward zero); and **tune λ by cross-validation** on a logarithmic grid, retraining on all training data at the winner. When to reach for which: Ridge when most features plausibly matter (dense truth) or as the default variance-control; Lasso when you believe few features matter and want the model to say which; Elastic Net when you want Lasso's selection but features correlate; and remember $\lambda \rightarrow 0$ recovers OLS while $\lambda \rightarrow \infty$ drives all (penalized) weights to zero — the full bias-variance dial of Chapter 4.

Logistic regression

Why "regression" for a classifier? Because logistic regression *is* a regression — on the log-odds. It models a continuous quantity, $\log \frac{p}{1-p}$, as a linear function of the features; classification only happens when you threshold the resulting probability. The name is historically accurate and conceptually helpful, and the one-line interview answer is: "it regresses the log-odds linearly; the class label is a decision rule bolted on afterward."

Build it from the need. We want $p(y = 1 | x)$ — a probability in $(0, 1)$ — from a linear score $z = w^\top x + b$ that lives in $(-\infty, \infty)$. The bridge is the **log-odds** (logit): odds $\frac{p}{1-p}$ map $(0, 1)$ to $(0, \infty)$, and the log maps that to all of $\mathbb{R}$. Setting

$$\log \frac{p}{1-p} = w^\top x + b$$

and inverting gives the **sigmoid**:

$$p(y = 1 | x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid facts worth having at recall speed: $\sigma(0) = 0.5$; $\sigma(-z) = 1 - \sigma(z)$; its derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, maximal at 0.25 when $z = 0$ and vanishing in the tails (the saturation behind vanishing gradients, Chapter 12); and it turns weight w_j into an **odds ratio**: one unit of x_j multiplies the odds by e^{w_j} — the interpretation used across medicine and credit scoring, and verified numerically in Listing 5.

The decision boundary is linear. Predicting class 1 when $p \geq 0.5$ is predicting when $z \geq 0$, and $w^\top x + b = 0$ is a hyperplane. The sigmoid is monotonic, so it never bends the boundary — it only calibrates distances from the hyperplane into probabilities. Logistic regression is a linear classifier; if the true boundary is a circle, it fails exactly the way linear regression fails on a curve, and the fix is the same (feature engineering — add x_1^2, x_2^2 and the "linear" boundary in the expanded space is a circle in the original). The threshold 0.5 is not sacred: it minimizes error only under symmetric costs, and moving it trades precision against recall (Chapter 10); with class imbalance or asymmetric costs, tuning the threshold on validation data is standard practice.

The loss. Squared error on probabilities is doubly wrong for classification: it is non-convex when composed with the sigmoid (bad optimization), and it barely punishes confident mistakes (bad statistics). Maximum likelihood gives the right loss. Each label is Bernoulli with parameter $\hat{p}_i = \sigma(w^\top x_i + b)$; the likelihood of the dataset is $\prod_i \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1 - y_i}$, and the negative log-likelihood, averaged, is **binary cross-entropy** (log loss):

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log (1 - \hat{p}_i)]$$

Cross-entropy punishes confident wrongness without bound — $\hat{p} \rightarrow 0$ on a true positive costs $-\log \hat{p} \rightarrow \infty$ — which is exactly the incentive a probability model should face. It is convex in w (a sum of log-sum-exp terms), so gradient descent finds the global optimum; there is **no closed form** (the sigmoid makes the stationarity condition transcendental), which is the standard answer to "why does linear regression have a normal equation and logistic doesn't?".

The gradient is the punchline. Differentiating through the sigmoid (chain rule, using $\sigma' = \sigma(1 - \sigma)$), almost everything cancels:

$$\nabla_w \mathcal{L} = \frac{1}{n} X^\top (\hat{p} - y)$$

— *identical in form* to linear regression's gradient with $\hat{p}$ in place of Xw : residual times features. Deriving this cancellation on a whiteboard is a top-five ML interview request, and the cleanliness is not an accident — it recurs for every GLM with its canonical link (next section) and for softmax + cross-entropy (Chapter 12). Listing 5 implements exactly this update and matches sklearn to three decimals.

Practical notes interviewers listen for: sklearn's LogisticRegression **applies L2 regularization by default** (parameter $C = 1/\lambda$, so *larger* C = *less* regularization — a perennial gotcha, and why Listing 5 sets $C = 10^{10}$ to compare against the unregularized scratch version). On **perfectly separable data**, unregularized weights diverge to infinity — the sigmoid can only reach its 0/1 targets in the limit $\|w\| \rightarrow \infty$, so regularization is not optional there. Probabilities from a well-fit logistic regression are typically decently **calibrated** (log loss is a proper scoring rule); heavily regularized or class-weighted fits

shift this, and Chapter 10 covers recalibration. Multiclass extension is **softmax regression** (multinomial logistic): K score vectors, softmax normalization, categorical cross-entropy — Chapter 12 treats it as the output layer of every classifier network.

Generalized Linear Models

Linear regression, logistic regression, and Poisson regression look like three models; GLMs reveal them as one recipe with three settings. The recipe has three components:

1. **A random component:** the target's conditional distribution, drawn from the exponential family — Gaussian, Bernoulli, Poisson, Gamma, and relatives, all sharing the form $f_Y(y; \theta, \phi) = \exp\left(\frac{y\theta - b(\theta)}{\phi} + c(y, \phi)\right)$ where θ is the natural parameter.
2. **A systematic component:** the linear predictor $z = w^\top x + b$.
3. **A link function g** connecting them: $g(E[y | x]) = z$, i.e. $E[y | x] = g^{-1}(w^\top x)$.

The link is chosen to map the mean's legal range onto all of $\mathbb{R}$. Gaussian mean lives in $\mathbb{R}$: identity link, and the GLM is linear regression. Bernoulli mean is a probability in $(0, 1)$: logit link, logistic regression — the log-odds construction above is exactly this step. Poisson mean is a positive rate: log link, Poisson regression, with the pleasant multiplicative reading that one unit of x_j *multiplies* the expected count by e^{w_j} .

Target	Distribution	Link $g(\mu)$	Inverse link	Model
Continuous, symmetric noise	Gaussian	identity	identity	Linear regression
Binary	Bernoulli	$\log \frac{\mu}{1-\mu}$	sigmoid	Logistic regression
Counts	Poisson	$\log \mu$	e^z	Poisson regression
Positive, right-skewed	Gamma	$\log \mu$ (common)	e^z	Gamma regression

Each listed link (identity, logit, log) is the distribution's **canonical link** — the one making the natural parameter itself linear in x — and canonical links are why the gradient miracle repeats: for every one of these models, the negative log-likelihood gradient is $\frac{1}{n} X^\top (\hat{\mu} - y)$, residual times features, with only the inverse link changing how $\hat{\mu}$ is computed. Listing 6 exploits this: the Poisson fitting loop differs from the logistic one by a single line (`np.exp` in place of `sigmoid`) and recovers the true parameters to three decimals, matching `sklearn`'s `PoissonRegressor`.

Why interviewers care, beyond taxonomy: GLMs are the right answer to "how would you model counts / rates / strictly positive targets?" — questions where naive linear regression embarrasses itself. Listing 6 shows linear regression on Poisson counts

predicting -5.34 events in a low-rate region; the GLM predicts 0.048 , a legal rate. GLMs also encode variance structure — Poisson variance *equals* the mean, so heteroscedasticity is built into the model rather than violating it (when real counts show variance exceeding the mean — "overdispersion" — negative binomial regression is the standard upgrade, a detail that reads as field experience). Fitting is by maximum likelihood, classically via iteratively reweighted least squares (IRLS — Newton's method that solves a weighted OLS problem per iteration), or plain gradient descent as in the listings. In production ML, GLMs survive as the giant sparse logistic regressions behind ad click-through prediction, insurance pricing (Poisson frequency $\times$ Gamma severity), and any setting where calibrated probabilities, convex training, and per-feature interpretability beat raw accuracy.

Code implementations

Every listing was executed as shown; outputs are real. Together they turn each claim in this chapter into a number: the normal equation agreeing with sklearn to machine precision, scaling deciding whether gradient descent converges at all, the validation minimum landing on the noise floor, Lasso's zeros, the logistic gradient's cancellation, the GLM recipe changing one line between models, and the two failure modes — collinearity and heteroscedasticity — measured rather than asserted.

Listing 1 — OLS three ways: normal equation, pseudoinverse, sklearn

Three routes to the same coefficients. The normal equation solves the linear system (never invert explicitly — `np.linalg.solve` is faster and more stable than computing an inverse); the pseudoinverse route runs on the SVD and survives singular designs; sklearn does SVD-based least squares internally. Agreement to $8.9\text{e-}16$ is machine epsilon: these are the same estimator.

```
"""Listing 1: OLS three ways -- normal equation, pseudoinverse, sklearn."""
import numpy as np
from sklearn.linear_model import LinearRegression

rng = np.random.default_rng(42)
n, d = 200, 3
X = rng.normal(size=(n, d))
true_w, true_b = np.array([2.0, -1.0, 0.5]), 4.0
y = X @ true_w + true_b + rng.normal(scale=0.5, size=n) # noisy linear truth

# Normal equation:  $w = (X'X)^{-1} X'y$  (solve, never invert explicitly)
Xb = np.hstack([np.ones((n, 1)), X]) # prepend bias column
w_ne = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)

# SVD pseudoinverse: numerically stable even when  $X'X$  is near-singular
w_pinv = np.linalg.pinv(Xb) @ y

lr = LinearRegression().fit(X, y)
w_sk = np.r_[lr.intercept_, lr.coef_]

print("truth          :", np.r_[true_b, true_w])
print("normal equation:", np.round(w_ne, 4))
print("pseudoinverse  :", np.round(w_pinv, 4))
```

```
print("sklearn      :", np.round(w_sk, 4))
print("max |diff|   :", f"{np.abs(w_ne - w_sk).max():.2e}")
```

Output:

```
truth      : [ 4.  2. -1.  0.5]
normal equation: [ 3.9815  2.0184 -1.0038  0.4742]
pseudoinverse : [ 3.9815  2.0184 -1.0038  0.4742]
sklearn     : [ 3.9815  2.0184 -1.0038  0.4742]
max |diff|  : 8.88e-16
```

Listing 2 — Gradient descent and the case for feature scaling

One feature has 100× the scale of the other. On raw features, the workable learning rates form a needle's eye: 1e-3 diverges to infinity, 1e-5 crawls (MSE 7.8 after 5,000 steps). Standardizing drops the condition number of the loss surface from 8,605 to 1.0, and the same problem converges to MSE 0.25 in a tenth of the steps at a 1,000× larger learning rate. The closed form, by contrast, is scale-indifferent — this is purely an optimizer problem.

```
"""Listing 2: gradient descent for OLS -- why feature scaling decides convergence."""
import numpy as np

rng = np.random.default_rng(0)
n = 500
x1 = rng.normal(0, 1, n)          # unit-scale feature
x2 = rng.normal(0, 100, n)        # feature 100x larger scale
X = np.c_[x1, x2]
y = 3 * x1 + 0.05 * x2 + rng.normal(0, 0.5, n)

def gd(X, y, lr, steps):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(steps):
        r = X @ w + b - y          # residuals
        w -= lr * (2 / n) * (X.T @ r) # dMSE/dw
        b -= lr * (2 / n) * r.sum()   # dMSE/db
        if not np.isfinite(w).all(): # diverged
            return w, b, np.inf
    return w, b, np.mean((X @ w + b - y) ** 2)

np.seterr(over="ignore", invalid="ignore") # divergence demo overflows on purpose
for lr_ in [1e-3, 1e-4, 1e-5]:
    w, b, mse = gd(X, y, lr_, 5000)
    print(f"raw      lr={lr_:<6g} MSE={mse:>10.4f}")

mu, sd = X.mean(0), X.std(0)
Xs = (X - mu) / sd                    # standardize
w, b, mse = gd(Xs, y, 0.1, 500)
print(f"scaled lr=0.1 MSE={mse:>10.4f} (500 steps, not 5000)")

# The culprit: condition number of X'X
for name, M in [("raw", X), ("scaled", Xs)]:
    ev = np.linalg.eigvalsh(M.T @ M / n)
    print(f"condition number ({name:6s}): {ev.max()/ev.min():.1f}")
```

Output:

```

raw      lr=0.001  MSE=      inf
raw      lr=0.0001 MSE=      1.4491
raw      lr=1e-05  MSE=      7.8486
scaled  lr=0.1    MSE=      0.2548 (500 steps, not 5000)
condition number (raw ): 8604.6
condition number (scaled): 1.0

```

Listing 3 — Polynomial degree sweep on a cubic truth

Noise variance is 9, so 9 is the unbeatable validation floor. Degree 3 essentially reaches it (9.13). Training error dropping *below* 9 (degrees 9–15) is noise memorization by definition — there is no legitimate signal left to fit. The pipeline standardizes the expanded features, without which high degrees would be numerically hopeless.

```

"""Listing 3: polynomial regression -- capacity sweep on a cubic truth."""
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(7)
def f(x): return 0.5 * x**3 - x**2 + 2 * x          # cubic ground truth
x_tr = rng.uniform(-3, 3, 40); y_tr = f(x_tr) + rng.normal(0, 3, 40)
x_va = rng.uniform(-3, 3, 200); y_va = f(x_va) + rng.normal(0, 3, 200)

print(f'{"deg":>3} {"train MSE":>10} {"val MSE":>10}')
for deg in [1, 2, 3, 5, 9, 15]:
    model = make_pipeline(PolynomialFeatures(deg), StandardScaler(),
                          LinearRegression())
    model.fit(x_tr[:, None], y_tr)
    tr = mean_squared_error(y_tr, model.predict(x_tr[:, None]))
    va = mean_squared_error(y_va, model.predict(x_va[:, None]))
    print(f'{"deg":>3} {"tr":>10.2f} {"va":>10.2f}')

```

Output:

deg	train MSE	val MSE
1	26.44	23.36
2	15.10	15.24
3	4.75	9.13
5	4.64	9.12
9	4.18	11.83
15	3.11	12.48

Listing 4 — Ridge, Lasso, Elastic Net on a sparse truth

Twenty features, five truly nonzero, and features 0 and 1 are near-duplicates (both with true weight 3). OLS handles the duplicate pair catastrophically — 0.05 and 6.07, an arbitrary split with the right sum. Ridge shares the weight evenly but keeps all 20 features. Lasso zeros 11 of the 15 true zeros but splits the correlated pair unevenly (2.62 / 3.36 — and the winner can flip on a re-drawn sample). Elastic Net delivers sparsity *and* an even split: the grouping effect, live.

```

"""Listing 4: Ridge vs Lasso vs Elastic Net on a sparse truth with correlated
features."""
import numpy as np
from sklearn.linear_model import Ridge, Lasso, ElasticNet, LinearRegression
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(1)
n, d = 100, 20
X = rng.normal(size=(n, d))
X[:, 1] = X[:, 0] + rng.normal(0, 0.05, n)      # feature 1 ~ duplicate of feature 0
true_w = np.zeros(d); true_w[[0, 1, 2, 3, 4]] = [3, 3, -2, 1.5, 1]  # 15 true zeros
y = X @ true_w + rng.normal(0, 1, n)
Xs = StandardScaler().fit_transform(X)

models = {"OLS": LinearRegression(),
          "Ridge(a=10)": Ridge(alpha=10),
          "Lasso(a=0.1)": Lasso(alpha=0.1),
          "ENet(a=0.1)": ElasticNet(alpha=0.1, l1_ratio=0.5)}
print(f"{'model':<13}{zeros':>6}{w0':>8}{w1':>8} (truth: w0=w1=3, 15 zeros)")
for name, m in models.items():
    m.fit(Xs, y)
    w = m.coef_
    print(f"{'name':<13}{np.sum(np.abs(w) < 1e-6):>6}{w[0]:>8.2f}{w[1]:>8.2f}")

```

Output:

model	zeros	w0	w1	(truth: w0=w1=3, 15 zeros)
OLS	0	0.05	6.07	
Ridge(a=10)	0	2.86	2.89	
Lasso(a=0.1)	11	2.62	3.36	
ENet(a=0.1)	7	2.92	2.96	

Listing 5 — Logistic regression from scratch

The whole training loop is the clean gradient: p minus y , times features. It matches sklearn (run nearly unregularized via $C=1e10$) to three decimals on both accuracy and coefficients. The last line checks the odds-ratio interpretation numerically.

```

"""Listing 5: logistic regression from scratch -- gradient descent on cross-entropy."""
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

rng = np.random.default_rng(3)
X, y = make_classification(n_samples=1000, n_features=5, n_informative=3,
                           n_redundant=0, random_state=3)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=3)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))

def fit_logistic(X, y, lr=0.1, steps=3000):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(steps):
        p = sigmoid(X @ w + b)      # predicted P(y=1|x)
        g = p - y                  # dL/dz -- the famous clean gradient
        w -= lr * (X.T @ g) / n
        b -= lr * g.mean()
    return w, b

```

```

w, b = fit_logistic(X_tr, y_tr)
p_te = sigmoid(X_te @ w + b)
acc_scratch = ((p_te >= 0.5) == y_te).mean()

sk = LogisticRegression(C=1e10, max_iter=5000).fit(X_tr, y_tr) # ~no regularization
acc_sk = sk.score(X_te, y_te)

print(f"scratch acc={acc_scratch:.4f} w={np.round(w, 3)}")
print(f"sklearn acc={acc_sk:.4f} w={np.round(sk.coef_[0], 3)}")
# log-odds sanity check: moving one unit along x0 multiplies the odds by e^{w0}
print(f"odds ratio for x0: e^{w[0]:.2f} = {np.exp(w[0]):.2f}")

```

Output:

```

scratch acc=0.8833 w=[-0.253  2.174 -0.07  0.85  0.377]
sklearn acc=0.8833 w=[-0.253  2.174 -0.069  0.851  0.377]
odds ratio for x0: e^{-0.25} = 0.78

```

Listing 6 — Poisson regression: the GLM recipe, one changed line

Compare `fit_poisson` here to `fit_logistic` above: the only substantive difference is `np.exp` where `sigmoid` was — the inverse link. Same clean gradient, same loop, different distribution. The scratch fit recovers the truth and matches `sklearn`'s `PoissonRegressor` to three decimals. The closer: linear regression on the same counts predicts -5.34 events for a low-rate input; rates cannot be negative, and the GLM's 0.048 is the kind of answer the problem actually admits.

```

"""Listing 6: GLMs -- Poisson regression from scratch vs linear regression on
counts."""
import numpy as np
from sklearn.linear_model import PoissonRegressor, LinearRegression

rng = np.random.default_rng(5)
n = 2000
X = rng.normal(size=(n, 2))
true_w, true_b = np.array([0.8, -0.5]), 0.3
lam = np.exp(X @ true_w + true_b) # log link: log E[y] = Xw + b
y = rng.poisson(lam) # count target

def fit_poisson(X, y, lr=0.05, steps=4000):
    n, d = X.shape
    w, b = np.zeros(d), 0.0
    for _ in range(steps):
        mu = np.exp(X @ w + b) # inverse link
        g = mu - y # same clean gradient form as logistic/OLS!
        w -= lr * (X.T @ g) / n
        b -= lr * g.mean()
    return w, b

w, b = fit_poisson(X, y)
print("truth      :", np.r_[true_b, true_w])
print("scratch GLM :", np.round(np.r_[b, w], 3))
sk = PoissonRegressor(alpha=0).fit(X, y)
print("sklearn     :", np.round(np.r_[sk.intercept_, sk.coef_], 3))

# Why not just linear regression? It happily predicts negative counts.
lin = LinearRegression().fit(X, y)
x_new = np.array([[-2.5, 2.5]]) # low-rate region

```

```
print(f"linear regression predicts {lin.predict(x_new)[0]:.2f} events (impossible)")
print(f"Poisson GLM predicts      {np.exp(x_new @ w + b)[0]:.3f} events")
```

Output:

```
truth      : [ 0.3  0.8 -0.5]
scratch GLM : [ 0.298  0.805 -0.531]
sklearn     : [ 0.299  0.805 -0.531]
linear regression predicts -5.34 events (impossible)
Poisson GLM predicts      0.048 events
```

Listing 7 — Multicollinearity: coefficient instability, measured

True weights are 1 and 1 throughout. As the correlation between the two features rises from 0 to 0.999, the OLS coefficient's standard deviation across re-drawn training sets explodes from 0.08 to 1.81 — at $\rho=0.999$ the fitted "effect of x_1 " routinely comes out negative or above 3, while predictions remain accurate (the *sum* along the shared direction is well determined; the split is not). Ridge caps the instability at every correlation level, at the price of mild shrinkage bias (mean 0.97 rather than 1.0).

```
"""Listing 7: multicollinearity -- coefficient instability, measured; ridge as the
fix."""
import numpy as np
from sklearn.linear_model import LinearRegression, Ridge

rng = np.random.default_rng(11)
n, trials = 150, 500

def coef_spread(rho, model):
    """Std of the first coefficient across re-drawn training sets."""
    w0 = []
    for _ in range(trials):
        x1 = rng.normal(size=n)
        x2 = rho * x1 + np.sqrt(1 - rho**2) * rng.normal(size=n) # corr(x1,x2)=rho
        X = np.c_[x1, x2]
        y = 1.0 * x1 + 1.0 * x2 + rng.normal(0, 1, n)
        w0.append(model.fit(X, y).coef_[0])
    return np.mean(w0), np.std(w0)

print(f"{'rho':>5} {'OLS mean w1':>12} {'OLS std':>8} {'Ridge mean w1':>14} {'Ridge
std':>10}")
for rho in [0.0, 0.9, 0.99, 0.999]:
    m_o, s_o = coef_spread(rho, LinearRegression())
    m_r, s_r = coef_spread(rho, Ridge(alpha=10))
    print(f"{'rho':>5} {'m_o':>12.3f} {'s_o':>8.3f} {'m_r':>14.3f} {'s_r':>10.3f}")
print("\nPredictions stay fine; it is the *coefficients* (and their story) that
break.")
```

Output:

rho	OLS mean w1	OLS std	Ridge mean w1	Ridge std
0.0	0.995	0.082	0.936	0.079
0.9	0.990	0.184	0.969	0.118
0.99	0.989	0.609	0.969	0.086
0.999	0.985	1.813	0.969	0.048

Predictions stay fine; it is the *coefficients* (and their story) that break.

Listing 8 — When OLS inference breaks: heteroscedasticity and coverage

The estimator stays unbiased under heteroscedasticity (slope mean 1.997 for a true 2) — that is not the problem. The problem is the *error bars*: the textbook standard-error formula assumes constant variance, and when error spread grows with x , the "95%" confidence interval covers the truth only 83% of the time — $3.4\times$ the promised error rate, silently. Fixes: robust (heteroscedasticity-consistent) standard errors, weighted least squares, or modeling the variance (a GLM view).

```

"""Listing 8: when OLS inference breaks -- standard errors under heteroscedasticity."""
import numpy as np

rng = np.random.default_rng(21)
n, sims = 200, 3000

def one_fit(hetero):
    x = rng.uniform(0, 4, n)
    noise_sd = 0.1 * np.exp(x) if hetero else 0.5 # error variance grows with x
    y = 1.0 + 2.0 * x + rng.normal(0, noise_sd, n)
    Xb = np.c_[np.ones(n), x]
    w = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)
    resid = y - Xb @ w
    s2 = resid @ resid / (n - 2) # homoscedastic sigma^2 estimate
    se = np.sqrt(s2 * np.linalg.inv(Xb.T @ Xb)[1, 1]) # textbook SE for slope
    covered = abs(w[1] - 2.0) < 1.96 * se # does the 95% CI contain truth?
    return w[1], covered

for hetero in [False, True]:
    slopes, cover = zip(*[one_fit(hetero) for _ in range(sims)])
    label = "heteroscedastic" if hetero else "homoscedastic "
    print(f"{label}: slope mean={np.mean(slopes):.3f} "
          f"95% CI coverage={np.mean(cover):.1%} (should be 95%)")

```

Output:

```

homoscedastic : slope mean=2.001 95% CI coverage=95.0% (should be 95%)
heteroscedastic: slope mean=1.997 95% CI coverage=83.0% (should be 95%)

```

Pitfalls, comparisons and practical tips

The comparison table interviewers expect you to have internalized:

	OLS	Ridge	Lasso	Elastic Net	Logistic
Penalty	none	L2	L1	L1 + L2	usually L2
Closed form	yes	yes	no	no	no
Exact zeros	no	no	yes	yes	with L1

	OLS	Ridge	Lasso	Elastic Net	Logistic
Correlated features	unstable split	even split	arbitrary pick	grouped	as penalty chooses
Works when $d > n$	no (non-unique)	yes	yes ($\leq n$ nonzero)	yes	with penalty
Bayesian prior	flat	Gaussian	Laplace	mixture	Gaussian (L2)

Normal equation vs gradient descent, the decision in one line each: closed form when d is modest and the loss is squared error — exact, no hyperparameters; gradient descent when d is huge, data streams or exceeds memory, or the loss is anything else. SGD (mini-batches, Chapter 13) is the industrial default for the latter.

Classic traps, and the tell for each:

- **Interpreting coefficients under collinearity.** "Feature X has a negative weight, so it hurts the target" — with correlated features, individual weights are nearly arbitrary (Listing 7's std of 1.81 on a true weight of 1). Check correlations or variance inflation factors before telling coefficient stories; prefer Ridge when you must interpret under correlation.
- **Regularizing unstandardized features.** The penalty then punishes features for their *units*, not their complexity. Standardize first, always; and don't penalize the intercept.
- **Tuning λ on the test set.** λ is a hyperparameter like any other — cross-validate it (Chapter 4's protocol). Reporting the CV-winning score as the final estimate is selection bias; use an untouched test set.
- **R^2 worship.** R^2 rises with every added feature, junk included; adjusted R^2 or held-out error are the honest versions (metrics in depth: Chapter 10). And a high R^2 says nothing about assumption validity — Anscombe's quartet has four identical R^2 values for wildly different data shapes; plot residuals.
- **sklearn's C in LogisticRegression.** It is *inverse* regularization strength, and regularization is *on by default* — a silent difference from statsmodels/textbook logistic regression that has burned many a coefficient comparison.
- **Squared-loss thinking on classification.** Fitting class labels 0/1 with linear regression produces predictions outside [0,1], a loss dominated by easy far-side points, and a boundary dragged by outliers. If asked "why not just linear-regress the labels?", those three failures are the answer, plus the non-convexity of squared loss through a sigmoid.
- **Trusting standard errors under violated assumptions.** Listing 8: unbiased slope, garbage confidence interval. If inference matters, check residuals and use robust standard errors or WLS.
- **Outliers under squared loss.** One point with a residual of 10 contributes as much loss as 100 points with residuals of 1. Options: investigate (it may be a data error —

Chapter 9), robust losses (Huber: quadratic near zero, linear in the tails; or absolute loss = median regression), or RANSAC for gross corruption.

- **Perfect separation in logistic regression.** Weights diverge, probabilities saturate to 0/1, and solvers warn or run to `max_iter`. Regularization fixes it; noticing it in the wild (a feature that perfectly encodes the label) is often a leakage alarm (Chapter 4).

Quick sanity habits. Fit an intercept-only baseline first (predicting the mean is $R^2 = 0$; anything worse than baseline means a bug). Compare train vs validation error before interpreting anything (Chapter 4's two-number diagnosis). For counts, rates, and strictly positive targets, reach for the matching GLM rather than transforming and hoping — and if you do log-transform the target, remember predictions back-transform with a bias correction (the mean of a log-normal is not the exponential of the mean of the logs).

Interview questions and answers

Q1. Derive the normal equation for linear regression.

Loss: $L(w) = (1/n)\|y - Xw\|^2$. Expand: $(1/n)(y^T y - 2w^T X^T y + w^T X^T X w)$. Gradient with respect to w : $(2/n)(X^T X w - X^T y)$. Set to zero: $X^T X w = X^T y$, so $\hat{w} = (X^T X)^{-1} X^T y$.

Convexity (the Hessian $(2/n)X^T X$ is positive semi-definite) makes the stationary point a global minimum. *Interviewers listen for: defining each matrix dimension, and mentioning the invertibility condition — $X^T X$ must be full rank, i.e. no perfectly collinear features.*

Q2. List the assumptions of linear regression and say which matter for prediction vs inference.

Linearity of the conditional mean, independent errors, homoscedasticity (constant error variance), normal errors — plus no perfect multicollinearity. For point prediction, linearity is the load-bearing one: OLS still gives the best linear approximation under violations of the rest. Independence, homoscedasticity, and normality matter for *inference* — standard errors, confidence intervals, p-values — which become miscalibrated when they fail (a nominal 95% CI covering 83% of the time in this chapter's Listing 8). Normality is the least critical: CLT rescues coefficient inference at large n .

Q3. Normal equation vs gradient descent — when would you use each, and what are the costs?

Normal equation: $O(nd^2 + d^3)$ — exact, hyperparameter-free, the right choice for modest d (up to a few thousand) with squared loss. Gradient descent: $O(nd)$ per iteration — the choice when d is very large, data doesn't fit memory or streams, or the loss has no closed form (logistic, most everything else). Practical additions: sklearn actually uses SVD-based least squares (more stable than forming $X^T X$, whose condition number is the square of X 's), and industrial scale uses SGD on mini-batches.

Q4. Why do we minimize squared error rather than absolute error? When would you prefer absolute error?

Squared error is the Gaussian MLE: assuming $y = w^T x + \varepsilon$ with Gaussian ε , maximizing likelihood is exactly minimizing squared error — plus it's smooth, convex, and admits a closed form. It estimates the conditional *mean*. Absolute error estimates the conditional *median*, has no closed form, and is robust to outliers — prefer it (or Huber loss, quadratic center with linear tails) when the target has heavy tails or gross errors, or when the median is the business-relevant summary. The deep version of the answer: choosing a loss is choosing a noise model and a target summary statistic.

Q5. Does gradient descent need feature scaling? Does the normal equation? Why the difference?

The normal equation is scale-indifferent — rescaling features rescales coefficients inversely and predictions are identical. Gradient descent's convergence rate is governed by the condition number of $X^T X$: wildly different feature scales elongate the loss bowl, forcing a learning rate small enough for the steepest direction and therefore a crawl along the shallowest. Listing 2 measures it: condition number 8,605 raw vs 1.0 standardized; divergence-or-crawl vs fast convergence. Regularized models add a second, independent reason to scale: the penalty compares coefficient magnitudes across features.

Q6. Why does Lasso produce exactly zero coefficients while Ridge only shrinks them?

Geometric answer: minimizing loss subject to a norm ball constraint — the L1 ball is a diamond with corners on the coordinate axes, and expanding elliptical loss contours preferentially touch corners, where coordinates are exactly zero; the L2 ball is a sphere with no special points, so tangency lands at generic nonzero coordinates. Gradient answer: L2's pull toward zero is proportional to the weight ($2\lambda w$ — fades as w shrinks, never finishes the job); L1's pull is constant λ regardless of size, so any weight whose data-gradient can't sustain λ of opposing force is pinned at exactly zero (soft thresholding). *Interviewers listen for: either picture told precisely; both is a strong answer.*

Q7. State the Bayesian interpretation of Ridge and Lasso.

Both are MAP estimation: the penalized loss is the negative log posterior with the penalty as negative log prior. Ridge = Gaussian prior $w \sim N(0, \tau^2 I)$ — smoothly discourages large weights, with $\lambda = \sigma^2/\tau^2$ connecting noise variance and prior width. Lasso = Laplace prior — sharply peaked at zero with heavier tails, encoding "most weights are exactly or nearly zero, a few may be large". This is Chapter 1's MLE-vs-MAP distinction realized: OLS is the flat-prior MLE, regularization is prior information.

Q8. You have 10,000 features, 500 samples, and you suspect only a few dozen features matter. Which regression do you fit and why?

OLS is unavailable ($d > n$ means $X^T X$ is singular; infinitely many perfect fits, none trustworthy). The sparsity belief points to L1: Lasso, or better Elastic Net since with 10,000 features many will be correlated and pure Lasso picks arbitrarily among correlated groups (and can select at most $n = 500$ features). Standardize features, tune λ (and the L1 ratio) by cross-validation — stratified/grouped as the data demands, per Chapter 4. Mentioning that the selected support is unstable under resampling — so downstream "the model chose these genes/features" stories need stability checks (e.g. selection frequency across bootstraps) — reads as real experience.

Q9. Why is logistic regression called regression if it's a classifier?

Because it literally is a regression — on the log-odds: $\log(p/(1-p)) = \mathbf{w}^T \mathbf{x} + b$ models a continuous quantity linearly. The output is a probability; classification appears only when you threshold it, and the threshold is a separate decision-theoretic choice (0.5 only under symmetric costs). Bonus depth: it's the Bernoulli GLM with the canonical logit link, sitting in the same family as linear (Gaussian, identity link) and Poisson (log link) regression.

Q10. Derive the gradient of logistic regression's loss and comment on its form.

Per-example cross-entropy: $L = -[y \log p + (1-y) \log(1-p)]$ with $p = \sigma(z)$, $z = \mathbf{w}^T \mathbf{x} + b$. Chain rule: $dL/dp = -y/p + (1-y)/(1-p)$; $dp/dz = p(1-p)$. Multiply: $dL/dz = p(1-p) \cdot [(p-y)/(p(1-p))] = p - y$. So $\nabla_{\mathbf{w}} L = (p - y)\mathbf{x}$ — residual times features, the identical form as linear regression's gradient. The cancellation is the signature of a canonical link paired with its exponential-family likelihood, and it recurs for softmax + cross-entropy. *Interviewers listen for: the clean execution of the cancellation; fumbling $d\sigma/dz = \sigma(1-\sigma)$ is the common stumble.*

Q11. Why not train logistic regression with squared error on the probabilities?

Two failures. Optimization: squared error composed with the sigmoid is non-convex — gradient descent can stall in flat regions (a confidently wrong prediction sits in the sigmoid's saturated tail where the squared-loss gradient is tiny). Statistics: squared loss caps the penalty for confident wrongness at 1, while cross-entropy sends it to infinity — the correct incentive for a probability forecaster, and the reason cross-entropy is the Bernoulli MLE while squared error corresponds to a Gaussian noise story that's false for binary outcomes.

Q12. What happens to unregularized logistic regression on linearly separable data?

Weights diverge: any separating hyperplane's loss keeps decreasing as $\|w\|$ grows (probabilities push toward 0/1 asymptotically, never arriving), so the MLE doesn't exist. Symptoms: solver max-iteration warnings, enormous coefficients, probabilities saturated at 0 and 1. Fix: any regularization makes the optimum finite. The field note that scores points: perfect separation on real data is often a leakage alarm — some feature encodes the label (Chapter 4).

Q13. Interpret a logistic regression coefficient of 0.7 on feature x_1 .

One unit increase in x_1 adds 0.7 to the log-odds, i.e. multiplies the odds of the positive class by $e^{0.7} \approx 2.01$ — doubled odds, holding other features fixed. Careful additions: this is *odds*, not probability (the probability change depends on where you start — biggest near $p = 0.5$, tiny in the tails); "holding others fixed" is unreliable under collinearity; and if features were standardized, the unit is one standard deviation.

Q14. What are the three components of a GLM? Map linear, logistic, and Poisson regression onto them.

(1) Random component: an exponential-family conditional distribution for y . (2) Systematic component: linear predictor $z = w^T x + b$. (3) Link function g with $g(E[y|x]) = z$. Linear regression: Gaussian + identity link. Logistic: Bernoulli + logit link. Poisson: Poisson + log link. The link maps the mean's legal range onto $\mathbb{R}$ (probabilities via logit, positive rates via log), and the listed links are canonical — which is why all three share the gradient form $X^T(\hat{\mu} - y)$.

Q15. You're predicting daily support-ticket counts. A colleague fits linear regression. Critique and propose better.

Counts are non-negative integers with variance that typically grows with the mean; linear regression can predict negative counts (Listing 6: -5.34 events), assumes constant variance, and models the wrong error distribution. Propose Poisson regression: log link keeps rates positive, coefficients read multiplicatively (e^w per unit feature), variance = mean is built in. Check for overdispersion (variance $>$ mean in residuals — common with bursty tickets); if present, negative binomial. If many exact zeros beyond what the rate explains (e.g. closed days), zero-inflated models. That escalation path — Poisson $\rightarrow$ overdispersion check $\rightarrow$ negative binomial — is the experienced answer.

Q16. Your linear regression's coefficients change wildly when you add a new feature, but validation error barely moves. Explain.

The new feature correlates strongly with existing ones. Along a shared direction, only the coefficient *sum* is well-determined by the data; the split among correlated features is nearly arbitrary and flips with small perturbations (Listing 7: coefficient std of 1.81 on a true weight of 1 at $\rho = 0.999$). Predictions live on the well-determined sum, so validation error is stable. Implications: don't tell causal or importance stories from these coefficients; use Ridge (stabilizes the split by preferring even sharing), drop redundant features, or examine VIFs. *Interviewers listen for: the distinction between prediction (fine) and interpretation (broken).*

Q17. What is polynomial regression, and why is it still "linear"?

Regression on expanded features ($x, x^2, \dots, x^k$) — the model remains linear *in the parameters*, so the normal equation, gradients, and all linear-model theory apply verbatim; only the feature map changed. It is the cleanest capacity dial: degree sweeps trace the full bias-variance curve (Listing 3: validation error falls to the noise floor at the true degree, then rises). Practical cautions: standardize expanded features (scale explosion), never extrapolate (polynomials diverge outside the data), and select degree by cross-validation, never training error.

Q18. How would you decide between Ridge, Lasso, and Elastic Net for a given problem?

By your belief about the true coefficient structure and your need for selection. Dense truth (many small effects — e.g. genomic polygenic scores, pixel weights): Ridge. Sparse truth, want interpretable feature selection: Lasso. Sparse-ish truth with correlated features (the common real case): Elastic Net — L1 for zeros, L2 for the grouping effect. Then let validation decide: cross-validate all three over λ grids; the metric differences are often small, and saying "I'd let CV arbitrate but expect X because Y" shows both priors and empiricism.

Q19. Why must features be standardized before regularized regression but not before OLS?

OLS is equivariant to affine feature rescaling: coefficients absorb the scale change, predictions unchanged. Penalties break this: $\lambda \Sigma w^2$ compares raw coefficient magnitudes, and a feature measured in grams gets a $1000\times$ smaller coefficient than the same feature in kilograms — so the penalty punishes the kilogram version $10^6\times$ less for identical predictive content. Standardizing puts all coefficients in per-standard-deviation units, making the penalty compare like with like. Related: never penalize the intercept — it encodes baseline level, not complexity.

Q20. Walk through what happens to Ridge solutions as λ goes from 0 to infinity. What about the bias-variance profile?

$\lambda = 0$: OLS — minimum-bias, maximum-variance member of the family. As λ grows, coefficients shrink monotonically toward zero (in the SVD basis, each component damped by $\sigma_i^2/(\sigma_i^2 + \lambda)$, weakest-data directions first); bias rises smoothly, variance falls. $\lambda \rightarrow \infty$: all penalized weights $\rightarrow 0$, model collapses to the intercept (prediction = mean). Validation error over λ is U-shaped, and the minimum shifts toward smaller λ as n grows — with more data you need less variance control. This is Chapter 4's regularization dial with an exact mechanism attached.

Q21. Implement one gradient-descent step for linear regression, by hand, on paper: X has rows (1, 0) and (1, 2) (intercept included), $y = (1, 3)$, w starts at (0, 0), $\eta = 0.1$.

Predictions: $Xw = (0, 0)$. Residuals $r = Xw - y = (-1, -3)$. Gradient $= (2/n)X^Tr = (2/2) \cdot ((1 \cdot -1 + 1 \cdot -3), (0 \cdot -1 + 2 \cdot -3)) = (-4, -6)$. Update: $w \leftarrow (0, 0) - 0.1 \cdot (-4, -6) = (0.4, 0.6)$. Sanity: both coordinates move toward the least-squares solution (1, 1). Interviewers listen for: correct residual direction (prediction minus target) and the transpose in X^Tr — the two classic sign/shape errors.

Q22. Your logistic regression outputs probabilities clustered near 0.5 with poor accuracy, but you know the features are informative. List plausible causes.

Ordered by likelihood: (1) over-regularization — sklearn's default $C=1.0$ can be strong for small or unscaled data; weights crushed toward zero push all probabilities toward 0.5; (2) unscaled features plus regularization — the penalty disproportionately kills large-scale features' coefficients; (3) the informative signal is nonlinear — a linear boundary through a circular pattern predicts ~ 0.5 everywhere; engineer interaction/quadratic features or move to a nonlinear model; (4) not converged — check `max_iter` warnings; (5) label noise or class overlap making 0.5 genuinely correct — check the Bayes rate with a strong nonlinear baseline.

Q23. What is heteroscedasticity, how do you detect it, and what does it break?

Non-constant error variance across the feature space — spread growing with predicted value is the classic pattern (incomes, prices). Detect: residuals-vs-fitted plot showing a funnel; formal tests (Breusch-Pagan). It does not bias coefficients (Listing 8: slope mean 1.997 for truth 2) — it breaks *inference*: textbook standard errors assume constant variance and become miscalibrated (measured 83% coverage for a "95%" CI). Fixes: heteroscedasticity-robust (sandwich) standard errors, weighted least squares if you can model the variance, or a GLM whose variance structure matches (Poisson/Gamma), or transform the target.

Q24. Why does sklearn's LogisticRegression sometimes give different coefficients than statsmodels' Logit on the same data?

sklearn regularizes by default (L2, $C = 1.0$; C is *inverse* strength), while statsmodels fits the unpenalized MLE. Shrunk coefficients differ from MLE coefficients — sometimes drastically for small n or unscaled features. To match: set C very large (or `penalty=None` in recent versions). Also worth naming: solver differences don't change the optimum (the loss is convex) but non-convergence does — check warnings. This is a real-world debugging classic disguised as an API question.

Q25. When $d > n$, what exactly goes wrong with OLS, and how does each regularizer resolve it?

$X^T X$ is at most rank $n < d$: singular, so the normal equation has infinitely many solutions — many fit training data perfectly and generalize arbitrarily badly. Ridge: $X^T X + \lambda I$ is full rank for any $\lambda > 0$ — unique solution, the minimum-norm-flavored one. Lasso: selects at most n features, yielding a unique sparse solution under mild conditions. Both encode the extra information (small or sparse weights) that the data alone cannot supply. Modern aside if invited: minimum-norm interpolation and double descent (Chapter 13) complicate the " $d > n$ is hopeless" story, but the classical answer stands for the classical regime.

Q26. The decision boundary of logistic regression is where $p = 0.5$. Show it's a hyperplane and explain when you'd move the threshold.

$p = \sigma(z) = 0.5$ exactly when $z = 0$, i.e. $w^T x + b = 0$ — a hyperplane, because σ is monotonic and only rescales distances from it into probabilities. Move the threshold under asymmetric costs or imbalance: threshold t corresponds to predicting positive when the odds exceed $t/(1-t)$, and the cost-optimal threshold is $C_{FP}/(C_{FP} + C_{FN})$ for misclassification costs C . Fraud with a 100:1 miss-to-false-alarm cost ratio wants a threshold near 0.01, not 0.5. Tune on validation data against the business metric (Chapter 10's precision-recall machinery).

Q27. Write the update loop for Poisson regression from scratch, and say what changes relative to logistic regression.

Loop: `mu = np.exp(X @ w + b); g = mu - y; w -= lr * X.T @ g / n; b -= lr * g.mean()`.

The only substantive change from logistic is the inverse link — `np.exp` where `sigmoid` was — because both are canonical-link GLMs sharing the gradient form $X^T(\hat{\mu} - y)$ (Listing 6 verifies against `sklearn` to three decimals). Numerical care: the exponential can overflow early in training — standardize features, start weights at zero, keep the learning rate modest or clip the linear predictor.

Q28. R^2 on train is 0.92; on validation it's 0.31. A teammate proposes adding polynomial features. React.

Oppose, with the diagnosis: a huge train-validation gap is high variance (Chapter 4's two-number rule) — the model already overfits, and polynomial features add capacity, making it worse. Prescriptions run the other direction: regularization (Ridge/Lasso with CV-tuned λ), fewer/better features, more data. Also audit for leakage (Chapter 4) — gaps this large in tabular problems are often a leaking feature rather than honest overfitting, and checking feature provenance costs nothing. Polynomial features become the right move only in the opposite regime: both scores low and close (high bias). *Interviewers listen for: diagnosis before prescription, and the leakage instinct.*